QUANTPHEMES MEETING REPORT

Q-Learning Paper-Trading Fleet for Hong Kong ETFs

A reproducible research-to-live workflow for training, evaluating, diagnosing, and deploying tabular reinforcement-learning strategies through the Quantphemes broker API.

Prepared for Quantphemes discussion | May 2026 | Public-shareable version with credentials and private identifiers removed

Python 3.11+ | Q-table RL | HK ETFs | YAML-driven experiments | Azure + systemd deployment

Executive Summary

Quantphemes RL is a compact but complete trading research platform. It starts from historical ETF data, trains tabular Q-learning agents under a strict fee and lot-size model, produces diagnostics and HTML reports, then deploys selected Q-tables to paper trading without retraining in production.

PAPER BOT FLEET

7

Five selected bots plus two imported collaborative 1-hour models integrated into the same live path.

ACTIVE ASSETS

3

2800.HK, 2828.HK, and 7226.HK. 7299.HK was halted after broker rejection.

EXPERIMENT SPACE

125+

Asset, reward, encoder, and action-space combinations tested in the 30-minute research grid.

CORE INVARIANT

20 bps

One-side fee, charged only on position changes and mirrored in training, oracle, and live sizing.

> **Meeting message:** the project is not just a bot script. It is a reproducible system: data adapters, experiment configs, plugin registry, trading environment, diagnostics, live execution, documentation, and deployment operations are all under version control.

GitHub repository: github.com/YelamanKarassay/Algo-Trading-RL
Documentation portal: yelamankarassay.github.io/Algo-Trading-RL/

Project Boundary

The project is currently a paper-trading and research system. Its main value is the repeatable pipeline: the same definitions of data, state, reward, action, fees, portfolio accounting, and model artifacts are used from historical training through live paper execution. This makes it easier to discuss results with Quantphemes

because every live action can be traced back to a configuration, a Q-table file, and a logged state tuple.

What Makes The Project Different From A Single Trading Script

Reproducible

Every experiment writes a config snapshot, git commit reference, metrics, logs, and trained Q-table. The user can rerun or compare experiments without remembering manual notebook state.

Extensible

New ideas are usually added as YAML files or small plugins. The core runner and live bot do not need a new if/else branch for every strategy.

Operational

Live paper trading uses systemd services, per-bot runtime state, structured JSON logs, and rollback-friendly artifacts rather than a laptop notebook session.

Paper-Trading Bot Portfolio

The deployed portfolio is intentionally diversified across assets, reward functions, encoders, and action spaces. This lets us observe whether behavior changes are caused by the model family, asset microstructure, state representation, or reward shaping.

BOT	ASSET	DECISION INTERVAL	REWARD / ENCODER	ACTIONS	BACKTEST ALPHA VS B&H	TRADES	FEES	STATUS
RL_CROSS	7226.HK	30 min	drawdown_penalty / three_feature	cash/full	+2.56%	30	HKD 59,795	paper running
RL_VOL	7226.HK	30 min	drawdown_penalty / with_volatility	cash/full	+2.56%	42	HKD 83,814	paper running
RL_FACOV	7226.HK	30 min	fee_aware / zscore_bins	cash/half/full	+2.56%	96	HKD 125,729	watch fees
RL_BROAD_A	2800.HK	30 min	fee_aware / zscore_bins	cash/half/full	+1.29%	10	HKD 9,902	paper running
RL_BROAD_B	2828.HK	30 min	log_return / with_volatility	cash/half/full	+1.30%	18	HKD 19,263	paper running
Collaborative HSI 1H	2800.HK	1 hour	QP-spec thresholds / three_feature	cash/full	-6.46 pp vs B&H	982 daily action trades	HKD 53,801	integrated
Collaborative Leverage 1H	7226.HK	1 hour	QP-spec thresholds / three_feature	cash/full	-3.56 pp vs B&H	576 daily action trades	HKD 29,372	integrated

How To Read Each Bot

RL_CROSS

This is the cleanest 30-minute 7226.HK leveraged-ETF cross-check. It uses the compact three-feature state and a drawdown-penalty reward with binary cash/full actions. The point of this bot is not complexity; it is a controlled leveraged baseline that answers: if we keep the state simple and penalize unstable intraday NAV paths, does the model still find moments where exposure is worth the fee?

It is useful in discussion because it is easy to explain. If this bot behaves sensibly, then the core state and reward timing are probably coherent. If it behaves poorly, the issue is less likely to be from exotic feature engineering and more likely from the basic signal or cost hurdle.

RL_VOL

This 30-minute 7226.HK bot keeps the same leveraged ETF setting but adds a volatility-regime encoder. It tests whether the policy should react differently during quiet periods versus active intraday periods. For a leveraged ETF this matters because the same raw price move can mean different things depending on the volatility environment.

The live observation question is whether volatility context makes the bot more selective. Ideally, it should avoid treating every small move as equal and instead condition exposure on whether the market is actually moving enough to justify risk and transaction cost.

RL_FACOV

This is the most experimental selected 30-minute 7226.HK candidate. It combines fee-aware reward, z-score state bins, and ternary actions: cash, half position, and full position. The model can scale exposure instead of making only all-or-nothing decisions, which is closer to how a human might reduce risk when the signal is not strong.

The reason it is marked as fee-watch is that the backtest produced the highest trade count and the highest fee bill among the selected models. That makes it informative but also risky: if live paper logs show frequent position changes without clear benefit, this is the first candidate to reduce or stop.

RL_BROAD_A

This 30-minute 2800.HK bot keeps the project connected to the original broad Hang Seng ETF objective. It uses fee-aware reward and z-score state bins, so price movement is interpreted relative to recent volatility rather than only through fixed thresholds.

It is important because 2800.HK is less leveraged and usually has smaller intraday movement. If the bot trades rarely or stays in cash, that may be rational after the 40 bps round-trip cost. In the fleet, this bot acts as a broad-market benchmark and a reality check against the more active leveraged ETF candidates.

RL_BROAD_B

This 30-minute 2828.HK bot adds a second broad ETF comparison point. It uses a log-return reward with a volatility-aware encoder and ternary actions. The goal is to see whether volatility-regime behavior generalizes beyond one asset and whether a non-7226 ETF can still produce useful intraday decisions after fees.

It also supports the federated-partner framing of the project: the same codebase can evaluate a related ETF universe without changing the training or live-bot architecture. That makes it a useful bridge between our internal tests and external collaboration.

Collaborative HSI 1H

This is a 1-hour 2800.HK Q-table imported from the collaborative artifacts and converted into our standard `QTableAgent` save format. It follows the simpler QP-spec setup: three-feature state, 27 states, and binary cash/full actions.

The provided walk-forward out-of-sample test covers 2022-05-13 to 2025-06-09, or 750 trading days. Net value declined from HKD 1,000,000 to HKD 952,271, a -4.77% net return after HKD 53,801 of fees. The strategy generated 491 round-trip trades, with a 24.24% win rate and profit factor of 0.327. Buy-and-hold net return over that different test window was +1.69%, so alpha versus buy-and-hold was -6.46 percentage points.

Its value is still comparison. It gives us an external 1-hour benchmark against our internally trained 30-minute models. The result also shows why transaction costs and trade frequency are central: the gross return was slightly positive, but fees turned the strategy negative.

Collaborative Leverage 1H

This is the leveraged counterpart to the collaborative HSI model: a 1-hour 7226.HK Q-table with the same simple three-feature binary structure. It lets us compare a slower, independently prepared leveraged policy against the 30-minute 7226.HK models trained in our research grid.

The provided walk-forward out-of-sample test covers 2024-04-05 to 2026-02-06, or 450 trading days. Net value declined from HKD 1,000,000 to HKD 968,787, a -3.12% net return after HKD 29,372 of fees. The strategy generated 288 round-trip trades, with a 15.97% win rate and profit factor of 0.164. Buy-and-hold net return over that separate window was +0.44%, so alpha versus buy-and-hold was -3.56 percentage points.

Because it uses the same broad artifact interface as our own agents, it can be deployed, logged, and monitored through the exact same live bot path. Analytically, it is not a same-window comparison with our 30-minute models, but it is a useful external stress test of the simpler 1-hour QP-spec policy.

Collaborative 1H Out-of-Sample Results

The two collaborative 1-hour strategies were tested on different historical windows, so their metrics should be read as contextual external benchmarks rather than direct apples-to-apples comparisons with the internal 30-minute research grid.

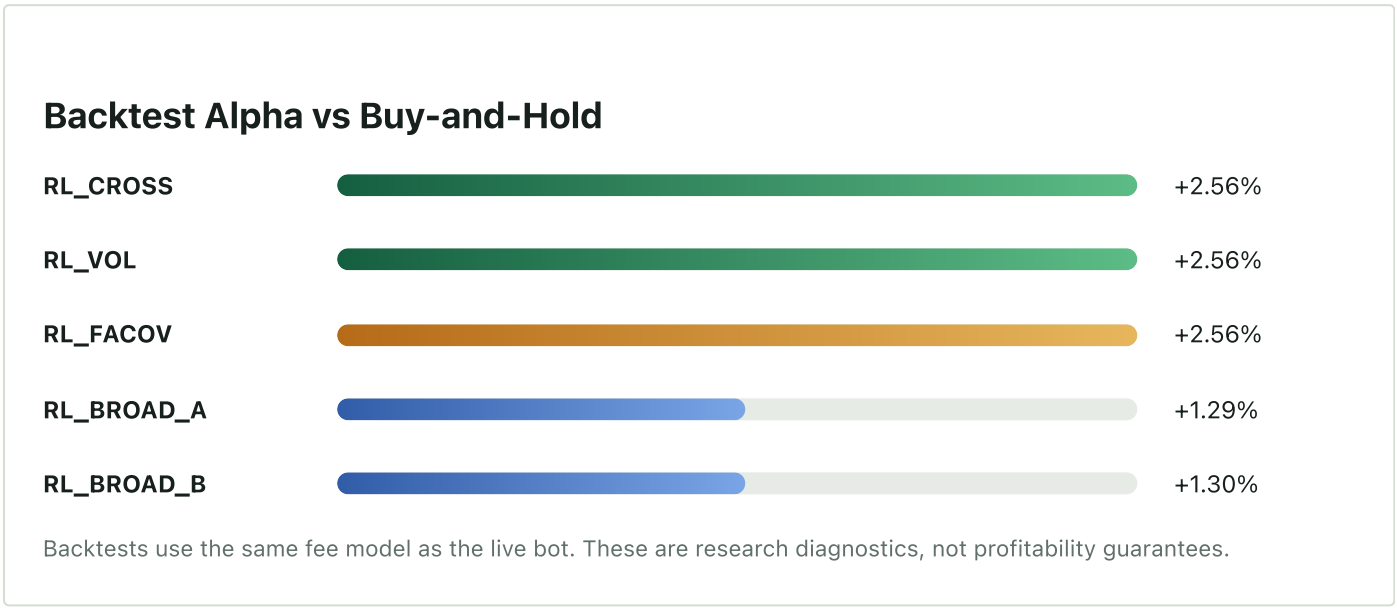
STRATEGY	TEST PERIOD	TRADING DAYS	END VALUE NET	NET RETURN	TOTAL FEES	ROUND TRIPS	WIN RATE	PROFIT FACTOR	ALPHA VS B&H
Collaborative HSI 1H	2022-05-13 → 2025-06-09	750	HKD 952,271	-4.77%	HKD 53,801	491	24.24%	0.327	-6.46 pp
Collaborative Leverage 1H	2024-04-05 → 2026-02-06	450	HKD 968,787	-3.12%	HKD 29,372	288	15.97%	0.164	-3.56 pp

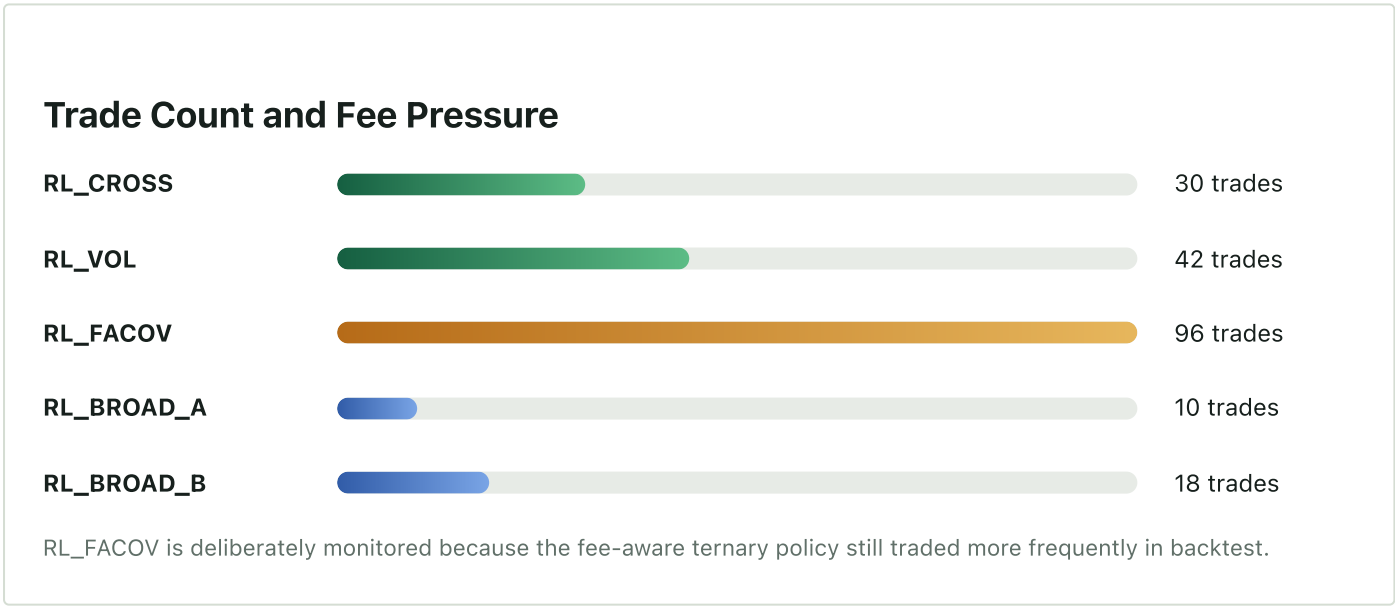
Both collaborative tests highlight the same engineering lesson that shaped our research grid: even when gross return is near flat or slightly positive, a high number of intraday trades can cause the net result to deteriorate after fees. This is one reason our selected internal paper fleet includes fee-aware rewards, z-score normalization, volatility features, and explicit monitoring of turnover.

Why The Fleet Uses More Leveraged ETF Candidates

The project started from 2800.HK because it is the natural broad Hang Seng ETF benchmark and a safer first production-style asset. However, the fee model is demanding: a full round trip costs around 40 bps. Many ordinary intraday moves in 2800.HK are smaller than that, so a Q-table can rationally learn to stay in cash or trade rarely.

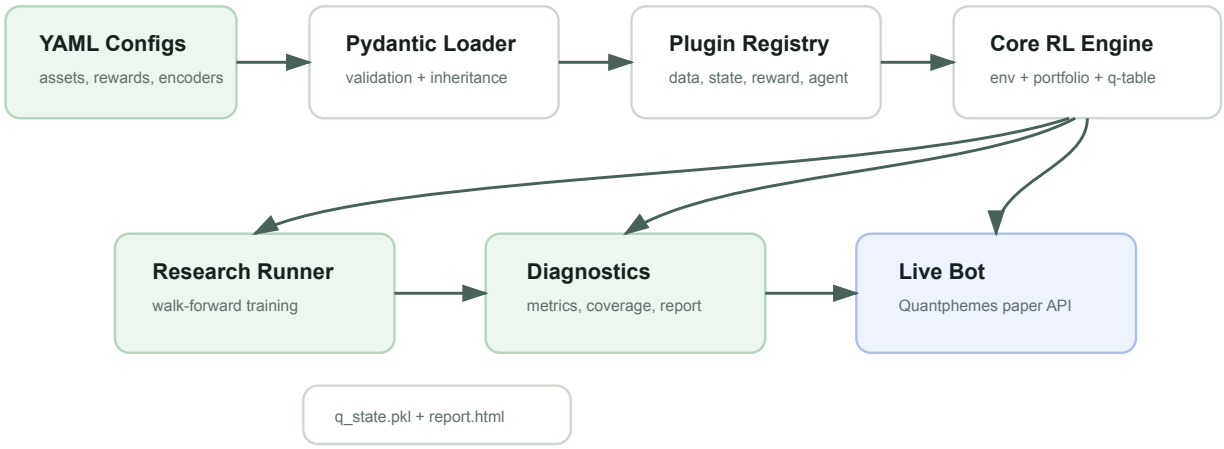
Leveraged ETFs such as 7226.HK can have larger intraday price movement, which gives the model more room to overcome transaction costs and makes state/reward differences easier to observe during paper trading. This does not mean leveraged ETFs are automatically better or safer. It means they are useful research instruments for testing whether the RL policy can identify enough movement to justify exposure after fees. The fleet therefore keeps 2800.HK as the broad benchmark while giving more paper slots to leveraged candidates where the signal-to-fee ratio may be more visible.





System Architecture

The project uses a shared core for both lab experiments and production trading. The live bot does not duplicate learning logic; it loads the same agent artifact created by the experiment runner and calls `Agent.act(state, training=False)`.



One architecture supports both research and production. This reduces drift between backtest logic and live behavior.

Why Pydantic + YAML?

YAML makes experiments readable and reviewable. Pydantic validates types, defaults, and inheritance so production can be configured without hidden code edits.

Why a Plugin Registry?

New data sources, state encoders, reward functions, and agents can be added without modifying the experiment runner or live bot control flow.

Why Tabular Q-learning?

The current state/action space is small enough for direct inspection. This makes early model behavior explainable before moving toward DQN or richer models.

Module Responsibilities

MODULE	PURPOSE	RATIONALE
`data/`	Normalizes Webull/Futu CSV data into HK-time bars; Bloomberg XLSX is supported as an adapter but was not the main source for this research batch.	Separates vendor formats from the RL environment.
`state/`	Encodes market context into finite discrete state IDs.	Preserves explainability and lets features be compared by plugin.
`reward/`	Converts NAV changes into learning signal.	Allows log-return, fee-aware, drawdown, and bias experiments.
`agent/`	Implements Q-table, baselines, persistence, and visit counts.	One artifact format supports training, diagnostics, federation, and live use.
`portfolio/`	Enforces cash, shares, fee, and lot-size accounting.	Ensures training and live sizing use the same financial constraints.
`env/`	Runs one trading day step-by-step.	Keeps reward timing and force-close fee handling testable.
`diagnostics/`	Builds coverage, oracle, signal, and HTML reports.	Explains why a model behaves as it does, not only what the NAV was.
`apps/`	CLI entry points for experiment, comparison, federation, and live bot.	Keeps executable workflows thin and auditable.

Why The Shared Core Matters

The most common failure mode in trading prototypes is backtest/live drift: the notebook tests one thing, while the production script implements a slightly different state, fee, timing, or sizing rule. This repository reduces that risk by centralizing the rules. The training environment, diagnostics, and live bot all use the same agent abstraction, state encoders, reward functions, and portfolio accounting model.

For Quantphemes review, this means a live paper decision can be explained in a deterministic chain: the bot reads the current price, builds a `MarketContext`, encodes it into a state index, reads the corresponding Q-table row for the current decision point, chooses the greedy action, converts that action into a target quantity using the broker cash and lot size, then patches the holding through the API.

Data, Configuration, and Plugin System

The system is designed so experiment behavior is visible in YAML rather than hidden in notebooks. Historical data, market assumptions, state encoder, reward function, agent class, training parameters, and evaluation settings are all declared in configuration files under `experiments/`. In practice, the research data used so far has mainly come from Webull and Futu exports/downloads. Bloomberg support exists in the codebase for future terminal exports, but it was not the primary data source for the current paper-bot fleet.

Accepted Data Inputs

- **Webull / generic CSV:** timestamp, open, high, low, close, volume. This is the main practical input path for the current 30-minute research batch after data is downloaded and saved under `data/raw/`.
- **Futu CSV:** time_key, open, close, high, low, volume, turnover. This is also part of the current practical data workflow and maps into the same internal `Bar` format.
- **Bloomberg XLSX:** supported for future use with Date, Open, High, Low, Last_Price, Volume, and case-insensitive field handling. It is included for compatibility but was not the main source for the current results.
- **Timezone rule:** all bars are normalized to Hong Kong time and grouped into trading days.

Data Validation Rules

- Required decision-time bars must exist for a day to be trainable.
- The HK lunch break is expected and is not treated as missing data.
- Final close handling matters because the strategy is day-trading and should be flat by end-of-day.
- Malformed vendor files raise errors early instead of silently entering training.

Configuration Model

CONFIG SECTION	CONTROLS	WHY IT IS EXPLICIT
`data`	Source, path, symbol, date range, train split.	Prevents ambiguity about which asset and historical period produced a Q-table.
`market`	Capital, fee, decision times, force close time, lot size.	Keeps backtest assumptions aligned with live trading constraints.
`state`	Encoder name and feature thresholds.	Makes feature changes auditable and prevents accidental baseline drift.
`reward`	Reward function and parameters.	Allows direct comparison of log-return, fee-aware, drawdown, and bias experiments.
`agent`	Agent type, action count, epsilon, learning mode.	Separates model behavior from market data and execution logic.
`artifacts`	Q-table path, runtime state path, log directory.	Lets multiple live bots run independently from the same repository.

YAML Reference Example

The example below shows the style of configuration used for a paper candidate. It is intentionally readable: a reviewer can see the asset, data file, interval, reward, encoder, action space, and live artifact paths without opening Python code.

```yaml
inherits: _baseline.yaml
name: production_rl_broad_a
description: 30-minute 2800.HK fee-aware z-score ternary paper strategy

data:
  source: csv
  path: data/raw/2800_hk_30m_webull_futu.csv
  symbol: 2800.HK
  start: 2018-01-01
  end: 2026-05-15

market:
  capital: 1000000
  fee_bps_one_side: 20
  decision_times: ["10:00", "10:30", "11:00", "11:30", "13:30", "14:00", "14:30", "15:00", "15:30"]
  force_close_time: "16:00"
  lot_size: 500

state:
  encoder: zscore_bins
  kwargs:
    lookback: 20

reward:
  function: fee_aware
  kwargs: {}

agent:
  type: qtable
  kwargs:
    num_actions: 3
    tie_break: random

artifacts:
  q_state_path: artifacts/deploy/rl_broad_a_q_state.pkl
  log_dir: artifacts/logs/rl_broad_a/
  runtime_state_path: artifacts/runtime_state_rl_broad_a.json
```

Plugin Registry

The registry is a small internal factory. Plugins register themselves by kind and name, for example `state_encoder: zscore_bins` or `reward_function: fee_aware`. The experiment runner and live bot call `build(kind, name, **kwargs)` instead of hard-coding classes. This is why we can run five internally trained bots plus two collaborative artifacts through the same production path.

Training, Evaluation, and Artifact Promotion

Training is handled by `apps.run_experiment`. It loads a YAML config, builds plugins through the registry, loads historical bars, groups bars into validated trading days, runs a walk-forward training loop, evaluates the greedy policy, writes diagnostics, and saves the trained Q-table as `q_state.pkl`.

Training Loop

- Each episode shuffles training days to reduce ordering bias.
- Each day starts with a fresh portfolio and zero position.
- At each decision point the agent acts, receives reward, and updates the Q-table.
- Epsilon decays after episodes, while visit counts preserve learning history.

Evaluation Loop

- Evaluation runs greedily with `training=False`.
- Baseline agents such as always-long, always-cash, and random can run alongside the main policy.
- Metrics include final NAV, alpha versus buy-and-hold, Sharpe, trades, fees, and state coverage.
- The generated report explains performance instead of only reporting a number.

Q-table Agent Mechanics

The Q-table has one table per decision point. A 5-decision binary baseline has shape `5 × 27 × 2`, while ternary models use three actions for cash, half position, and full position. The agent stores both Q-values and visit counts. Visit counts drive per-cell learning rates and later diagnostics, including coverage heatmaps and federated merge weighting.

One hard rule is that exact Q-value ties break randomly. This prevents a silent default-to-cash bias in unvisited states. That rule matters because many live states may initially be sparse, especially when a richer encoder expands the number of possible bins.

Promotion To Paper Trading

A model is promoted by copying a selected `q_state.pkl` into the configured artifact path for a production YAML. No retraining happens in the live bot. The live bot only loads the artifact and acts greedily. This keeps production behavior stable and makes rollback simple: restore the previous artifact and restart the affected service.

Research Methodology

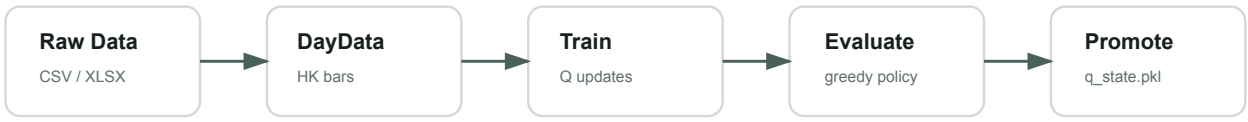
The research grid compares assets, reward functions, state encoders, and action spaces under the same transaction-cost model. This gives us a controlled way to decide which behaviors are worth deploying to paper trading.

Model Family

- One Q-table per decision point.
- Binary actions: cash or full position.
- Ternary actions: cash, half, or full position.
- Random tie-breaks to avoid a hidden cash bias.

Experiment Controls

- Starting capital: HKD 1,000,000.
- One-side fee: 20 bps.
- Fresh portfolio each simulated day.
- Flat by end-of-day in live operation.



Why the Current Five Were Selected

The selected paper set is a monitoring portfolio rather than a final production claim. It includes broad ETF candidates with lower fee load, leveraged ETF candidates with more movement, and feature variants that allow us to observe live sensitivity to volatility and z-score regimes.

Reward Function Rationale

REWARD	PURPOSE	INTERPRETATION
`log_return`	Pure baseline reward from portfolio value change.	Simple, finance-standard, and directly tied to compounded performance.
`log_return_long_bias`	Tests whether the agent was too reluctant to take exposure.	Useful for behavior discovery, but it can increase fee spend if overused.
`fee_aware`	Adds stronger pressure against unnecessary position changes.	Important because round-trip cost is large relative to many intraday ETF moves.
`drawdown_penalty`	Penalizes poor intraday NAV dips.	Designed to reduce policies that only look good at the close but suffer unstable paths.
`sparse_liquidation`	Concentrates reward around realized liquidation outcomes.	Useful as a research contrast, but often increases noisy turnover.

State Encoder Rationale

ENCODER	FEATURES	REASON FOR TESTING
`three_feature`	Move vs yesterday close, today open, and previous decision price.	Compact baseline matching the original QP-spec 27-state setup.
`zscore_bins`	Price moves normalized by recent volatility.	Attempts to compare moves across assets with different volatility scales.
`with_volatility`	Baseline price state plus realized volatility regime.	Tests whether the policy should behave differently in quiet versus active intraday markets.
`with_rsi`	Adds momentum/overbought-oversold proxy.	Tests whether short-term reversal or trend context improves decisions.
`with_volume`	Adds relative volume or spike context.	Tests whether liquidity/activity changes help distinguish meaningful price moves.

Live Operations

Production-style paper trading runs on Azure using `systemd`. Each bot has a separate service, config, Q-table artifact, runtime state file, and JSONL log directory. Secrets are loaded from server-side environment variables and never committed to Git.

Safety

Dry-run mode is available for testing. Live mode only patches broker holdings when dry-run is omitted and strategy/portfolio IDs are present.

Monitoring

Every decision emits structured JSON with timestamp, state tuple, action, price, target quantity, current quantity, and status.

Rollback

Services can be stopped independently; Q-table artifacts can be restored from prior promoted files without changing source code.

Known Platform Notes

- `7299.HK` was rejected by the broker as not in the tradable stock list, so related bots were halted.
- Price reads may succeed for a symbol even when trade PATCHes are rejected. Tradeability must be verified separately.
- Live bot force-close behavior explicitly targets zero quantity near market close; the broker still determines actual execution and paper-trade visibility.

Live Decision Lifecycle

STAGE	WHAT HAPPENS	AUDIT TRAIL
Morning setup	The bot waits for the HK session, fetches or records the open, and loads runtime state.	`runtime_state_*.json`
Decision time	The bot fetches current price, builds state, chooses action, and computes target quantity.	JSON decision log with state tuple and action.
Broker sync	The bot reads cash/current quantity and patches target holdings when live mode is enabled.	Quantphemes API response and bot status field.
Heartbeat	If action is unchanged, the bot can still send a same-target patch as a heartbeat.	Status value distinguishes heartbeat from target change.
Close	The bot targets zero quantity near close and records the close for tomorrow's previous close.	Force-close log entry and runtime state update.

Collaborative Model Integration

The two collaborative ZIP artifacts contained standalone bot scripts and Q-table pickle files. Their Q-tables were structurally compatible with the repository agent format: `5 × 27 × 2` tables, visit counts, and the same three-feature threshold structure. They were converted into the standard `QTableAgent.save` artifact format so they can run through the same `apps.bot` live path as the internally trained models.

During that integration, a live-state bug was fixed: `delta_prev` must use the previous decision price, not the current price. Without that fix, the third feature would be artificially neutral during live trading. The fix benefits both internal and collaborative Q-tables.

Documentation and Engineering Discipline

The project includes a MkDocs Material knowledge base with architecture, data guide, experiment workflow, live runbook, API integration notes, research notes, and developer guide. This is designed for mixed readers: Quantphemes reviewers, teammates, supervisors, and future maintainers.

DOCUMENT AREA	WHAT IT EXPLAINS
Architecture	How configs, plugins, environments, agents, diagnostics, and live execution fit together.
Data Guide	Required fields, Webull/Futu CSV workflow, supported Bloomberg import format, HK lunch break, 16:00 close, and timezone pitfalls.
Runbook	Environment variables, dry-run/live commands, Azure deployment, systemd monitoring, rollback.
Developer Guide	How to add plugins and tests without breaking the registry or production path.

Quality Controls and Current Boundaries

The repository includes automated tests for registry behavior, data adapters, portfolio accounting, trading environment reward timing, Q-table updates, baselines, config loading, experiment execution, reports, live-bot logic, runtime state, and federated merge behavior. The most important trading invariants are covered by regression tests because small logic errors can dominate strategy outcomes.

Automated Quality Gates

- `ruff check .` for linting and import hygiene.
- `pytest -q` for unit and smoke tests.
- `mkdocs build --strict` for documentation link/build validation.
- GitHub Actions runs CI and documentation builds on the repository.

Current Boundaries

- Tabular models are explainable but limited; they cannot learn richer nonlinear patterns.
- Intraday ETF edges are small relative to the 40 bps round-trip cost.
- Paper execution depends on Quantphemes symbol whitelist and platform plan constraints.
- Backtest alpha versus buy-and-hold should be validated over more market regimes.

What The Report Does Not Claim

This report does not claim that the current paper strategies are production-profitable. It claims that the system for researching, diagnosing, deploying, and monitoring Q-table strategies is implemented and auditable. Live paper results should be interpreted together with broker execution behavior, fee load, market regime, and symbol tradeability.

Quantphemes Discussion Points

- Confirm the official tradable universe for HK ETFs and leveraged ETFs.
- Clarify how paper holding PATCHes map to visible orders, fills, and position updates in the dashboard.
- Confirm plan-level constraints around strategy automation, paper portfolios, and API request limits.
- Confirm best-practice endpoint usage for intraday prices versus trading/holding updates.

Prepared for Quantphemes discussion. Generated from repository state and local experiment outputs. No API keys, portfolio IDs, or private credentials are included.